Compaction Marks Hard Tasks: A Causal Test of Context Summarization in a Gemma 4 Coding Agent

Delaying or removing ADK's summarizer left the score unchanged; firing it earlier cost ten tasks

Apoorv Saraogee

Independent researcher, San Jose, CA, USA

ORCID 0009-0002-9137-2725

Gemma 4 Developer Agent Paper Track, Kaggle, November 2026

Abstract

In the Gemma 4 Developer Agent harness, Google ADK summarizes the conversation once a request passes 14,336 tokens. The summarizer reads only text, so every earlier tool call and tool output is lost and the issue statement is paraphrased. Here I test whether this compaction causes the failures it is associated with. Across 35 full runs of Gemma 4 31B on 125 public tasks, compaction fired in 79% of 4,375 task runs, and runs that compacted solved 33% of tasks against 70% for runs that did not. However, within the same task compaction lowers the solve rate by only 2.2 points (95% CI −4.6 to +0.1), which indicates that the gap is mostly a marker of task difficulty. A threshold intervention gave the same answer: at 20,480 tokens, compaction fell from 78% to about half of tasks, and two runs scored 56 and 56 against 55 and 55. With the summarizer off, one run scored 56, although a quarter of tasks then overflowed the context window. In contrast, at 10,240 tokens tasks averaged two summaries, and two runs scored 46 and 43, a loss of 10.5 tasks (95% CI 4.0 to 17.0). The logged summaries show why the default is benign: the model's narration carries the target file into 82% of summaries, and three bundle-side remedies did not help. A pre-registered replication on 121 tasks from 33 other repositories met all four predictions: delaying compaction cut its share from 81% to 46% of tasks and moved the solved count by +5.0 tasks (95% CI −0.8 to +10.8). These results suggest that length-triggered events in agent traces should be analyzed within task or by intervention.

1 Introduction

The competition fixes everything except the agent bundle. The model is Gemma 4 31B with 4-bit weights, served by vLLM with a 32,768-token window, and the agent gets 28 tool calls and 8 minutes per task. The scorer sets ADK's event compaction (summarization of the conversation history) to a 14,336-token threshold. A bundle can change the system prompt, sampling, tools, skills and LoRA adapters, but it cannot change compaction.

Compaction is the most visible failure correlate in my traces. A run that compacts loses every file body, search hit and test output it has seen, along with the verbatim issue. The natural reading is that compaction causes failures, and that a bundle should fight it with shorter reads, pinned notes or re-grounding rules. Hossain reported the same association in this competition, while noting that it is not causal [1].

However, compaction is triggered by the length of a run, and length is itself an outcome. A run that is going badly searches longer, reads more and thinks more, so it crosses the threshold more often. A comparison of compacted with uncompacted runs therefore mixes the effect of compaction with the reasons a run grew long.

The objective of this study is to separate the two. I measure compaction in 4,375 task runs and validate the measure against logged summaries. To remove between-task difficulty, I compare runs within the same task. I then move the threshold while holding the agent fixed, read the summaries themselves, test three bundle-side remedies, and repeat the threshold test on held-out repositories under a pre-registered plan.

2 Setup

Harness and model. I run the competition harness (swegemma 0.2.11, adk_submission 0.2.13, google-adk 1.36.1) with vLLM 0.19.1 and the scorer's model, serving length and 8,192-token output cap. Per-task time limits are scaled by each pod's measured decode speed, by factors of 1.26 to 1.86.

Tasks. The 129 public tasks come from fastapi, rich, requests and httpx. The reference sandbox lacks test dependencies that many of them import [9]. Mine installs them, and I keep a task only if it passes with the reference patch and fails with an empty one, which leaves 125 tasks for all comparisons.

Compaction mechanics. Before each model call, ADK reads the prompt size the server reported for the previous call. At 14,336 tokens or more, it replaces all but the most recent events with a summary. Retention keeps calls paired with their responses, so the last three tool calls and their outputs survive verbatim. The summarizer formats each older event as `author: text`. That admits the agent's visible text and reasoning but no tool calls or tool outputs, which I confirmed from logged summarizer inputs. The summary returns as a user message beginning "For context:". The first user message, which holds the issue, always falls inside the summarized range. The system instruction is rebuilt from session state on every call and is never summarized.

Measurement. Each run writes a trace per task with per-call prompt tokens and timestamps. I mark a compaction where the prompt falls below 70% of the previous call after that call exceeded 9,000 tokens. To validate the rule, I patched my local runner to log every summary with its input. In five logged runs, the rule and the log agree on whether a task compacted for 610 of 625 tasks. The rule finds 96% of logged summaries at the default and delayed thresholds and 82% at 10,240, so counts of repeat summaries are lower bounds.

Statistics. Paired comparisons on the same 125 tasks use the exact McNemar test. Within-task effects use Mantel-Haenszel estimates with tasks as strata. Contrasts between settings average each task over the runs of an arm and treat the 125 task differences as the sample.

3 The observational picture

Table 1 pools 35 full runs at the scorer's threshold. The bundles differ in prompt and sampling, while the model, harness and tasks are fixed.

Table 1: Compaction across 4,375 task runs at the default threshold

Quantity	Value
Task runs that compact	79% (46% to 87% per run)
First compaction, tool call	median 14 of 28 (interquartile range 11 to 18)
Solved when compacted	1,137 of 3,465 (33%)
Solved when not compacted	639 of 910 (70%)
Compacted before the first edit: solved	223 of 1,483 (15%)
Compacted after the first edit: solved	914 of 1,982 (46%)
Median duration of a turn that compacts	44 to 65 s per run, against 4 to 8 s otherwise

The raw gap is 37 points, and it is larger still before the first edit.

However, 87 of the 125 tasks compacted in some runs and not in others, and within those tasks compacted runs solve only 2.2 points less often (Mantel-Haenszel risk difference, 95% CI −4.6 to +0.1; odds ratio 0.72, CI 0.49 to 1.06). By count, one compaction gives an odds ratio of 0.79 and two or more give 0.55, against Hossain's 0.50 and 0.39 with a leave-one-out difficulty control [1]. Nearly all of the 37-point gap therefore lies between tasks, because hard tasks both compact and fail. The small residual likely overstates harm, because within a task a failing run still tends to run longer.

4 A threshold intervention

Because observational controls cannot remove that last confounder, I moved compaction while holding the agent fixed. My runner exposes the scorer's threshold as a flag and can switch compaction off. I ran my best local bundle twice at each of three thresholds and once with no summarizer.

Table 2: Same bundle at four compaction settings

Threshold	Solved of 125	Tasks compacted	First compaction, call	Summaries per compacted task	Timeouts	Window overflows
10,240	46, 43	95%, 90%	10, 10	2.2, 1.9	31, 58	0, 0
14,336 (scorer)	55, 55	77%, 78%	15, 14	1.3, 1.4	11, 17	0, 1
20,480	56, 56	51%, 46%	21, 21	1.0, 1.1	19, 14	0, 2
Off	56	none	n/a	n/a	2	30

At 20,480 tokens, about a third fewer tasks compacted, compaction came six to seven calls later, and second summaries nearly vanished, from 28 and 34 tasks to 2 and 4. The score did not change: 56.0 tasks on average against 55.0 (difference +1.0, 95% CI −4.2 to +6.2).

The sharpest test is the set of tasks the intervention switched off. In the first delayed run, 26 tasks compacted in both default runs and not here, and they were solved 11 and 11 times by default and 11 when delayed. In the second, 32 such tasks were solved 15 and 13 times by default and 13 when delayed. Inside the delayed runs the familiar gap reappears, 30% solved when compacted against

59% when not, now drawn around different tasks. This suggests that the gap follows the length of the run rather than the summary.

As a limiting case, one run with the summarizer removed solved 56 (difference +1.0, 95% CI −4.7 to +6.7), and timeouts fell to 2. Instead, 30 tasks ended when the prompt outgrew the context window. By default those 30 tasks compacted in all but one case and were solved 4 and 5 times. The other 95 tasks were solved 53 times without a summarizer, against 51 and 50 with it. The summarizer carries a quarter of tasks past the window, and those tasks mostly fail either way.

In contrast, lowering the threshold to 10,240 tokens reduced the score. Nine tasks in ten compacted, about twice each, starting at call 10, and the two runs solved 46 and 43, a loss of 10.5 tasks (95% CI 4.0 to 17.0; sign test $p = 0.002$). The loss was concentrated in tasks with repeated summaries. Tasks that compacted at least twice were solved 24 and 17 times, against 30 to 32 and 24 to 25 for the same tasks by default. Each summary is a slow turn (Table 1), and timeouts rose from 11 and 17 to 31 and 58, with nine of the 10.5 lost tasks falling among the 62 tasks that timed out in an early-threshold run. Tasks that compacted only at the lower threshold lost little: 14 of 16 and 10 of 14 were solved, against 14 and 12 by default.

The response to the threshold is therefore not gradual: from 14,336 tokens up to no summarizer, totals stay within one task, whereas below it compaction turns from a marker into a cost.

5 What the summary keeps

The logged summaries explain why the default is benign. A summary runs about 3,000 characters, written from about 9,500 characters of input. That input is the agent's own narration, and Gemma narrates as it works: "Let's examine the code around line 1563 in `fastapi/applications.py`." The narration names what the agent was reading, so 82% of 1,085 summaries name a file from the reference patch.

However, a summary cannot carry file content, and after a compaction 82% of `read_file` calls reopen a file read before it. The share of `edit_file` calls rejected because `old_string` was not found doubles, from 4% to 8%. The agent pays these costs in tool calls and, by Section 4, usually recovers.

A summarizer that never sees a tool call can also misstate progress. Of 343 first summaries written after the agent had already edited a file, 35 say the fix has not been applied. Those tasks were solved 14 times in 35, against 149 in 308 for the rest, although the edits themselves remain in the working tree.

6 Bundle-side remedies

Prior work favors state that lives outside the summary [2, 3]. ADK 1.36.1 gives a bundle one such place: session state, which the system instruction renders on every call. I tested three remedies, each a single change.

Table 3: Remedies on the same base bundle (default threshold)

Bundle change	Solved of 125	Did the mechanism fire?
Pin the issue through a `{problem_description}` placeholder in the system prompt	55, 50 (default 55, 55)	Yes. Prompts grow 0.30 tokens per issue character ($r = 0.995$)
Pin plus a note tool: an AgentTool child whose `output_key` fills `{notes?}`	52 ($p \geq 0.63$ against either pin run)	Partly. Notes in 66% of tasks, before the first edit in 47% of those
Pin plus a rule for the "For context:" message: check `git diff`, re-read exact lines	49 ($p \geq 0.15$ against either pin run)	Rarely. `git diff` within two calls of 6% of compactions, against 2% to 3% without the rule

None of the three remedies improved the score, and the pin, which delivered the verbatim issue to every call, scored 2.5 tasks below the default (95% CI −7.6 to +2.6). On the competition's hidden leaderboard split, the pinned bundle scored 0.15 and the base bundle 0.15 and 0.13. The note tool was used, but late, and its extra tokens raised the share of compacting tasks from 78% to 88%, whereas the re-grounding rule was mostly ignored. These results suggest that Gemma 4 uses state that arrives verbatim but does not reliably act on rules that a later message is meant to trigger.

7 Held-out repositories

The competition is scored on private repositories [9], whereas my 125 tasks come from four public ones. I therefore pre-registered a replication on 121 tasks from 33 other repositories in FreshSWE-Py [11], using a second bundle at 14,336 and at 20,480 tokens (two runs each). The plan, the analysis script and the task list were frozen in the linked dataset at 03:29 UTC on 12 November, before the runs began, and the plan discloses a 20-task pilot.

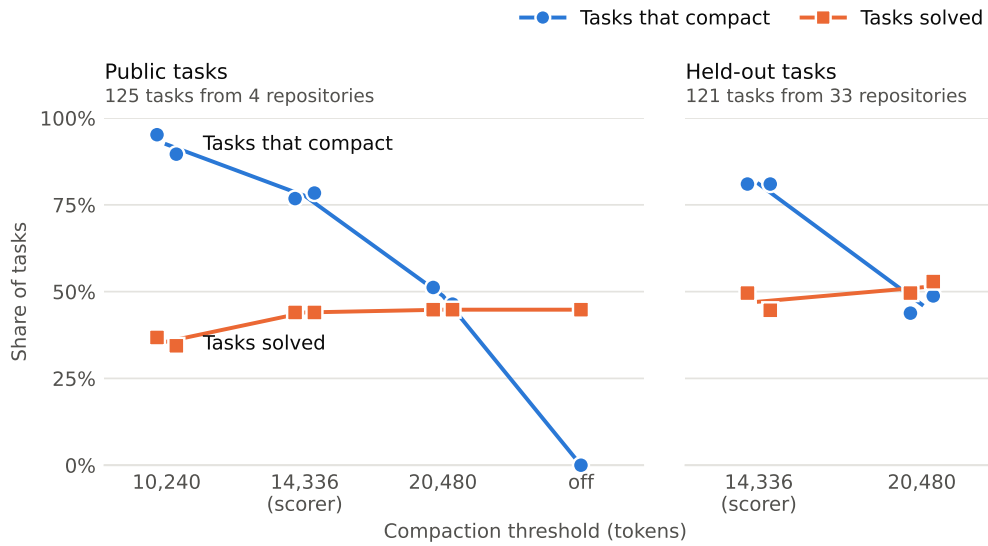


Figure 1: Tasks that compact and tasks solved at each threshold, public and held-out. Each point is one run.

All four registered predictions held (Figure 1). The default runs solved 60 and 54 tasks with

compaction in 81% of tasks, and the delayed runs solved 60 and 64 with compaction in 44% and 49%. The registered contrast is +5.0 tasks (95% CI −0.8 to +10.8; 16 tasks better, 8 worse), inside the registered bound of six tasks but close to it, and both registered sensitivity analyses agree. The 21 tasks that compacted in both default runs and in neither delayed run were solved 11 and 11 times by default and 10 and 10 when delayed. In an exploratory split, the gain lies among the 73 tasks that still compacted in a delayed run (from 22.0 to 28.0 solved per run), which leaves a benefit of later summaries possible. The observational gap is as large as on the public tasks: across four default-threshold runs of three bundles, compacted runs solved 38% of tasks against 82%, whereas within the 29 tasks that did both they solved 1.0 points more often (95% CI −6.5 to +8.5). One deviation from the plan: that interval uses Sato's sparse-data variance [6], because the registered variance assumes large strata and gave −2.3 to +4.3.

8 Noise

Four pairs of identical runs flip the outcome of 11% to 19% of tasks, while their totals differ by 0 to 5. Across the 35 default runs, totals have a standard deviation of 3.1 tasks. With 14 to 24 tasks flipping by chance, a paired exact test needs a net gain of 11 to 13 tasks to reach $p < 0.05$. On the hidden leaderboard split, the second bundle scored 0.20, 0.13 and 0.17 in identical submissions, and 0.15 at a higher temperature.

9 Related work

Context management for coding agents. Lindenbauer et al. found that masking old observations matched LLM summarization on SWE-bench Verified at about half the cost of raw history [4]. Satish et al. varied what, when and how much to compress across nearly 35,000 runs of three open models [5]. For Qwen3.5-35B-A3B, raising the trigger from 10K to 20K tokens raised summary success from 50.7% to 62.7%. I see the same low point at 10K but no reliable gain from 14K to 20K. Their summarizers read tool results and keep the task verbatim. ADK 1.36.1 does neither, so its summaries depend on how much the agent narrates.

What summaries lose. Compact-Bench finds that LLM summaries keep at most 16.0% of the details that recur later in a repair [7], and Wang et al. found that compactors keep 17% of injected side constraints [2]. ADK's maintainers later added tool calls and tool responses to the summarizer input after an ablation "found those carry the analysis and evidence a summary must preserve" [8]. For a model that narrates its work, my results suggest the text channel already carries the anchor, and lost tool output can be re-read.

This competition. A concurrent pre-registered study asks whether an agent that has already edited loses that state at compaction [10].

10 Limitations

The interventions use one model and one harness, with two runs per threshold on each task set and one without a summarizer. Results on this scale exclude large effects only: the interval for delaying compaction reaches a gain of 6 tasks on the public tasks and 11 on the held-out ones. At 10,240

tokens I cannot separate the time a summary takes from the information it drops, and a timeout is itself an outcome of a run going badly. My clock is emulated on local GPUs and its tightness varies by pod, so timeout counts compare only roughly across runs.

11 Conclusion

At the scorer's threshold, compaction marks hard tasks more than it breaks them. Raising the threshold removed it from more than a quarter of tasks, switching it off removed it from all, and neither changed the score. A pre-registered test on 33 other repositories met all four predictions, with a non-significant gain of 5.0 tasks. In contrast, lowering the threshold so that summaries came early and often cost about ten tasks. At 14,336 tokens there is therefore little left for a bundle to recover. More broadly, any event that fires when a trace grows long will correlate with failure, so analysts of agent traces should compare runs within a task or move the trigger and observe what changes.

Reproducibility. A notebook recomputes every table and the held-out analysis from per-task records and documents the runner flag and the summary logger: kaggle.com/code/apoorvsara/compaction-marks-hard-tasks-paper-tables. Its dataset holds 5,996 task-run records, features of 1,085 logged summaries, and the frozen plan with its analysis script.

References

[1] S. M. Hossain. "Too Small to See: Resolution Limits, Leaky Statements and Re-sent Reasoning When Tuning a Gemma 4 31B Software-Engineering Agent." Gemma 4 Developer Agent Paper Track, Kaggle, 2026. Notebook: kaggle.com/code/mobassir/why-your-leaderboard-can-t-see-a-3-point-gain

[2] Z. Wang et al. "Lost in Context Compaction: When LLM Systems Drop Side Constraints." arXiv:2608.11242, 2026.

[3] P. Rajasekaran et al. "Effective context engineering for AI agents." Anthropic Engineering, September 2025.

[4] T. Lindenbauer et al. "The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management." arXiv:2508.21433, 2025.

[5] R. Satish et al. "Beyond Token Savings: A Systematic Study of Context Compression in LLM Agents." arXiv:2609.32961, 2026.

[6] T. Sato. "On the variance estimator for the Mantel-Haenszel risk difference." Biometrics 45(4):1323-1324, 1989.

[7] R. Sun et al. "Compact-Bench: Measuring What Context Compaction Costs Long-Horizon Coding Agents." arXiv:2610.31710, 2026.

[8] google/adk-python commit bdb5582, "feat: include thoughts and tool calls in compaction summaries."

[9] J. Wold. "27 fastapi tasks cannot pass in the provided sandbox (missing inline_snapshot and dirty_equals)." Paper Track discussion 753370, Kaggle, 2026.

[10] pedronog8. "Pre-registration: edit-state loss at context compaction (confirmatory test on held-out repos)." Paper Track discussion 756850, Kaggle, 2026.

[11] H. Mittelman. "FreshSWE-Py: post-release Python tasks from 33 repositories." Kaggle dataset moshegolderson/freshswe-py, 2026.